

Language Evaluation and Design

The objective of this chapter is to discuss the relation between a modeling language and a set of real-world phenomena that this language is supposed to represent. By exploring this relation, we propose a systematic way to design and evaluate *modeling languages* with the objective of reinforcing truthfulness to the corresponding domain in reality and conceptual clarity of the specifications produced using those languages.

In section 2.1, we discuss the various aspects that comprise a system of representations, or simply, a language. In that background section, we briefly discuss the issues of (abstract and concrete) syntax, (formal and real-world) semantics and pragmatics.

In section 2.2, we concentrate on the definition of an evaluation framework that can be used to systematically assess the *real-world semantics* of an artificial modeling language, i.e. how suitable a modeling language is to model phenomena according to a given real-world conceptualization.

Since conceptualizations are abstract entities, in order to precisely conduct the evaluation framework advocated in section 2.2, a concrete representation of a conceptualization must be made available. In this thesis, a domain conceptualization is expressed in terms of a shared conceptual specification of the domain, named here a *domain ontology*. In section 2.3, we discuss the role that ontologies of material domains (such as law, medicine, archaeology, genetics) play in formalizing the semantic domain of domain-specific languages and informing properties that can be exploited in the design of efficient visual pragmatics for these languages. In particular, we illustrate our approach with an example in the domain of genealogy. Since one of our main objectives in this thesis is to evaluate and improve the quality of *general conceptual modeling languages*, in section 2.3 we also discuss the characteristics that are required for a domain independent meta-conceptualization that to which a general conceptual modeling language should commit.

In section 2.4, we discuss the role that explicit represented domain ontologies can play in semantically interoperating (e.g., integrating, comparing, translating) models produced in languages that diverge in syntax and semantics but whose underlying real-world conceptualizations overlap. This topic, albeit related, falls outside the central concern of this thesis. For this reason, it is only briefly discussed.

In section 2.5, we present some final considerations.

2.1 Elements of Language Design

According to (Morris, 1938) a language comprises three parts: **syntax**, **semantics** and **pragmatics**. Syntax is devoted to “*the formal relation of signs to one another*”, semantics to “*the relation of signs to real world entities they represent*” and pragmatics to “*the relation of signs to (human) interpreters*”. In the following subsections we elaborate upon these three definitions.

2.1.1 Syntax

In (Harel & Rumpe, 2000), the authors discuss the distinction between the purist notion of information and its syntactical representation as data, which is the medium used to communicate and store information. Data is essential to give a concrete and persistent status to some information, but it is in itself, however, vacuous in terms of meaning. Thus, to extract the information behind a piece of data, an **interpretation** is necessary that assigns meaning to it. The same piece of information may be represented as different data (e.g., “*December 31st, 2002*” and “*the last day of the year 2002*” refer to the same entity). Likewise, the same piece of data may serve as a **representation** for different things for different people at different points in time. The difference between data and information resembles the difference between a language syntax and semantics, respectively.

In order to communicate, agents must agree on a common communication language. This fixes the sets of signs that can be exchanged (syntax) and how these signs can be combined in order to form valid expressions in the language (syntactical rules). In sentential languages, the syntax is first defined in terms of an alphabet (set of characters) that can be grouped into valid sequences forming words. This is called a lexical layer and it is typically defined using regular expressions. Words can be grouped into sentences according to precisely defined rules defined in a context-free grammar, resulting in an abstract syntax tree. Finally, these sentences are constrained by given context conditions. The list of valid words of a language is called its *vocabulary*.

In diagrammatic (graphical) languages, conversely, the vocabulary of the language is not defined in terms of linear sequence of characters but in terms of pictorial signs. The set of available graphic modeling primitives forms the lexical layer (the *concrete syntax*) and the language's *abstract syntax* is typically defined in terms of an abstract visual graph (Erwig, 1999) or a metamodel specification. The latter alternative applies to the OMG's Model Driven Architecture (MDA) initiative (Bézivin & Gerbé, 2001) and it is the one that is considered in this thesis. Finally, the language metamodel specification is enriched by context conditions given in some constraint description language, such as, OCL or first-order logic (FOL). In either case, context conditions are intended to constrain the language syntax by defining the set of correct (well-formed) sentences of the language. Some of these constraints are motivated by semantic considerations (laws of the domain being modeled) while others can be motivated by pragmatic issues (discussed in section 2.1.3).

In summary, in the context of this thesis, a *language metamodel specification* (defining the abstract syntax of a language) defines the rules for the creation of well-formed models in that language. In other words, it defines the set of grammatically correct models that can be constructed using that language. In contrast, the vocabulary, or concrete syntax of that language, provides a concrete representational system for expressing the elements of that metamodel.

2.1.2 Semantics

Besides agreeing on a common vocabulary, participants in a communication process must share the same meaning for the syntactical constructs being communicated, i.e., they must interpret in a compatible way the expressions of the communication language being used. Without a proper way to extract information, the data being exchanged is worthless. Semantics deals with the meaning of signs and symbols or, in other words, with the information behind a piece of data.

Although most authors agree that semantics concern the relation between the expressions of a language and their meanings, opinions diverge when explaining the nature of this relation. As a consequence, there is a long-standing philosophical dispute concerning the meaning of "meaning" (Gärdenfors, 2000). A major divergence on the ontological status of the semantic relation concerns whether semantics is *referential* or not. In referential semantics, there are some kinds of objects (things in the world or mental entities) that are the meanings of linguistic expressions. Within the area of philosophy of language there is also a *functionalist* tradition of meaning which is non-referential. According to (Gärdenfors, *ibid.*), the most well-known proponent of this view is the later Wittgenstein, who

defended the view that is often summarized by the slogan “*meaning is use*”, i.e., that the (linguistic) meaning of a linguistic expression is its (canonical, proper) communicative function, i.e., its potential contribution to the communicative function of utterances of which it forms part. To put it simply, in a functionalist approach the meaning of a word is not given by a reference to an extra-linguistic entity, but by a set of circumstances and intentions in which the word is used.

From now on we concentrate on the referential approach, due to its suitability to the enterprise pursued in this thesis and its compatibility with the semantic tradition of fact-based conceptual modeling languages (Hirschheim, Klein & Lyytinen, 1995), which are the focus of this work.

In general terms, a semantic definition for a language $\mathcal{L}$ consists of two parts: (i) a semantic domain D ; (ii) a semantic mapping (or interpretation function) from the syntax to the semantic domain, formally $I: \mathcal{L} \rightarrow \mathcal{D}$ (Harel & Rumpe, 2000). The semantic mapping tells us about the meaning of each of the language’s expressions in terms of an element of the semantic domain.

In computer science, the relation between the semantic domain and the domain conceptualization can be interpreted according to two different stances to semantics. In AI research, semantics denote “*some form of correspondence specified between a surrogate and its intended referent in the world; the correspondence is the semantics for the representation*” (Davis & Shrobe & Szolovits, 1993). In other words, semantics is a mapping (interpretation) from the language vocabulary to concepts that stand for entities in the real world. Conversely, approaches with heritage in other traditions of programming consider semantics without a commitment to a specific real-world conceptualization. In these approaches, the term “semantics” is used to denote rules for program compilation or automated interpretation. According to these approaches, the semantic domain is a mathematical domain not necessarily related to a real-world conceptualization. In the AI approach, conversely, the semantic domain is typically a *material domain*, such as, engineering, business, medicine or telematics. In (Ferreira Pires, 1994; Vissers & van Sinderen & Ferreira Pires, 1993), these are called *formal* and *architectural semantics*, respectively. From now on, we use the term *domain conceptualization* when referring to a real-world conceptualization and the term *mathematical conceptualization* when referring to purely mathematical one. Architectural semantics, under the term ***real-world semantics*** (Partridge, 2002; Sheth & Kashyap, 1992; Vermeer, 1997; Vermeer & Apers, 1996), have been perceived as fundamental for semantically interoperability and semantic integration of information sources. In (Partridge, 2002), for instance, it is claimed that: “*Underlying the variety of forms of integrating data and applications there is a common semantic task - what can be called the ‘semantic matching’. There is a reasonably clear recognition*

that the analysis stage of this task needs to focus on identifying the entities that the data describes, i.e. the ‘real-world semantics’”.

In (Ferreira Pires, 1994; Visser & van Sinderen & Ferreira Pires, 1993), it is argued that designers should concentrate on the elaboration of models, using the modeling language merely as a vehicle for the representation of design characteristics. A modeling language can only be useful for its community of users, if its vocabulary, syntax, semantic and pragmatics, are defined based on the needs of this community for the elaboration of specifications. This view, supported here, emphasizes the precedence of real-world concepts over mathematical concepts in the design and evaluation of modeling languages or, in other words, the precedence of real-world semantics over purely mathematical semantics.

In section 2.2 we discuss in detail the relation between a language (as a set of interrelated modeling primitives) and a domain conceptualization, in the definition of the real-world semantics of this language.

2.1.3 Pragmatics

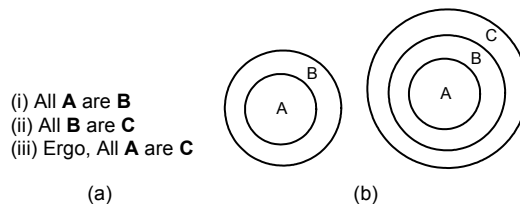
Following Morris’ definition (Morris, 1938), pragmatics concerns “*the relation of signs to (human) interpreters*”. Human users have contact with a modeling language (and with specifications produced in this language) via the language’s concrete syntax.

A system of concrete syntax may be considered to be a collection of objects and some relations between these objects. The type (visual or otherwise) of a particular representation, and more generally of a language, is determined by the characteristics of the *symbols* used to express these objects and relations (Gurr, 1999). Throughout this discussion we use the term symbol in a broad sense. Thus, our symbol definition includes visual features (e.g., morphological, geometric, spatial and topological relations) used by diagrammatic languages to represent objects and relations.

In sentential languages, there is a clear separation between vocabulary, syntax and semantics. The syntactic rules which permit construction of sentences, may be completely independent of the chosen vocabulary, and may be clearly distinguished from a definition of semantics. For example, let P be a propositional logic whose vocabulary consists of the propositions p and q , and the symbols $\wedge$ and $\neg$ representing the logical connectives ‘and’ and ‘not’, respectively. The syntactic and semantic rules for P tell us, respectively, how to construct and interpret formulas using this vocabulary. However, we may substitute the symbols $\{p, q, \wedge, \neg\}$ for $\{X, Y, \&, \sim\}$ throughout P to produce a logic which is effectively equivalent to P . Alternatively, we could retain the vocabulary and syntax of P , while altering the semantics to produce a vastly different logic (ibid.).

The same does not hold for graphical languages, in which vocabulary, syntax and semantics are not clearly separable. For example, a graphical vocabulary may include shapes such as circles, squares, arcs and arrows, all of differing sizes and colors. These objects often fall naturally into a hierarchical typing which almost certainly constrains the syntax and, furthermore, informs about the semantics of the system. Likewise, spatial relations, such as inclusion, are part of the vocabulary but clearly constrain the construction of potential diagrams and are likely to be mapped onto semantic relations with similar logical properties (ibid.). This idea is illustrated by Figure 2.1 below, in which two different languages are used to express logical syllogisms. The sentential language of (Figure 2.1.a) and the visual language of (Figure 2.1.b) are semantically equivalent. Despite that, the inference step that culminates with conclusion (iii) is performed in a much more straightforward way in the language of Euler's circles (figure 2.1.b). This classic example shows how semantic information can be directly captured in a visual symbol. Here a sequence of valid operations is performed which cause some consequence to become manifest in a diagram, where that consequence is not explicitly insisted upon by the operations. This is because the partial order properties of the set inclusion relation are expressed via the similarly transitive, irreflexive and asymmetric visual properties of proper spatial inclusion in the plane, i.e. the representing relation has the same semantic properties as the represented relation. This argument is given a formal account by (Shimojima, 1996) where immediate inferences like in (Figure 2.1.b) are termed "*free-rides*".

Figure 2-1 Logical Syllogism represented in a sentential language (a) and in the visual language of Euler's Circles (b)



In sentential languages, the relationships between symbols are necessarily captured in terms of the concatenation relation, which must then be interpreted by some intermediate abstract syntax (Gurr, 1999). In visual languages, intrinsic properties of the representation system can be systematically used to directly correspond to properties in the represented domain. Gurr uses the term *directness* to denote the correspondence between properties of representations and the properties of which they represent, and the term *systematicity* to denote the systematic application of directness in the design of systems of concrete syntax.

A notion of key importance to the discussion promoted here is the one of *implicature* proposed by the philosopher of language H. P. Grice (Grice, 1975, 1978). An implicature is anything that is inferred from an utterance but that is not a condition for the truth of the utterance. Grice distinguishes between the meaning of a sentence and what is implied by it. For instance, the utterance “*John has 14 children*” commonly implicates “*John has precisely 14 children*”, even though it would be compatible with John having 20 children. Likewise, the sentences “*You should work as well as study*” and “*You should study as well as work*”, despite of perhaps being semantically equivalent, can implicate different things. A conversational implicature is founded on the assumption that conversational participants will adhere to, what Grice names, a *cooperative principle*. According to Grice, the cooperative principle governs informative conversational discourse and is based upon the assumption that conversational participants adhere to certain rules, named *conversational maxims*, and, as consequence, that they will construct and recognize utterances as carrying intended implied information and, avoiding unwanted implications. Conversational maxims state that a speaker is assumed to make contributions in a dialogue which are *relevant, clear, unambiguous, brief, not overly informative and true according to the speaker's knowledge*. When a statement breaches any one of these maxims, additional meaning is inferred. To illustrate this idea the following dialogue is presented:

A. *How are you doing?*

B. *Still alive.*

In this example, the conversational maxim of relevance is broken. This leads to the interpretation *by implicature* – in addition to the obvious interpretation that the person is alive – that the person is not in good health or is experiencing some difficulties.

The relevance of notion such as *systematicity, implicatures, and conversational maxims* to the design of (visual) modeling languages can be motivated as follows. When correctly employed, systematicity can result in major increases in the effectiveness of the diagram for performing specific tasks. For instance, in (Hahn & Kim, 1999), the authors show how three semantically equivalent languages, namely, UML activity, sequence and collaboration diagrams can differ drastically in terms of effectiveness when employed for model integration. In contrast, whenever misused or neglected, implicatures can communicate incorrect information and induce the user to make incorrect inferences about the semantics of the domain. To illustrate this idea, we show in figure 2.2, 2.3 and 2.4 three versions of the same diagram. This example is cited by (Gurr, 1999) and has been

taken from (Marks & Reiter, 1990). Figure 2.2 represents a computer's disk subsystem. The visual concrete syntax also expresses relations and intrinsic properties of represented domain entities via either icons or other visual features such as the use of spatial inclusion within a box drawn with a dotted line denoting membership of a subsystem. In figure 2.3, the inherent ordering of graphical symbols used to represent the nodes can be incorrectly interpreted that all nodes in the represented network fall into a single conceptual category, in which they are similarly ordered. In figure 2.4 we can find several examples of directly inferred incorrect information: (i) one of the device queues in the upper section of the diagram has been laid out irregularly which can be interpreted that this queue must be different than the others somehow; (ii) in the lower section of the diagram the disk-symbols are organized as two separated groups, wrongly implicating that this division reflects some grouping in the domain; (iii) the line-width used for the lower queue in the channel facility differs from the one used for all other queues in the diagram; (iv) a different font has been used for the channel facility's text label, wrongly implying that it must have different subsystem status.

Figure 2-2
Representation of a
computer's disk
subsystem

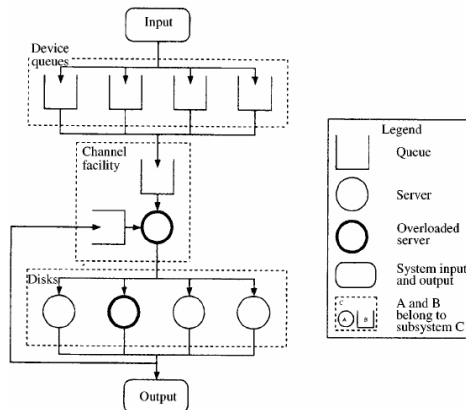


Figure 2-3 Semantically equivalent alternative representation of the model depicted in figure 2-2

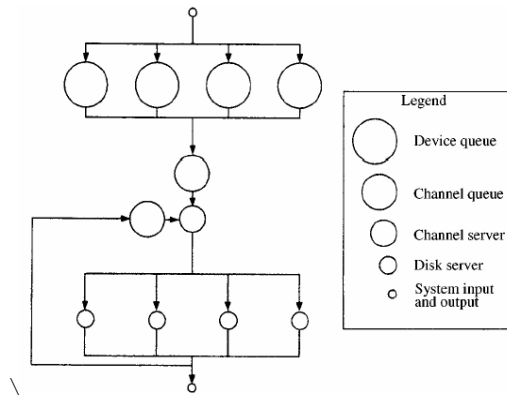
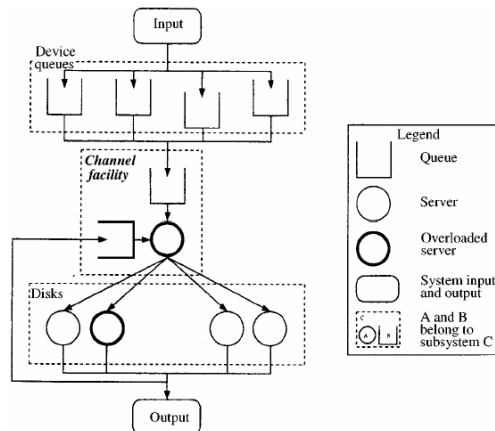


Figure 2-4 Semantically equivalent alternative representation of the model depicted in figure 2-2



All three diagrams of Figures 2.2, 2.3 and 2.4 faithfully portray all the features of the assumed network model, i.e., diagrams 2.3 and 2.4 cannot be considered misleading through purely semantic considerations. The problematic implications of these diagrams do not follow from the incorrect use of the symbols but rather by various aspects of its semantics which are not explicitly specified, but that can be misleading (Gurr, 1999).

While examples of Figures 2.3 and 2.4 illustrate the potential traps of ignoring pragmatic aspects of diagrams, other research has shown that the efficacy of diagrams for communicating information can be increased by the correct usage of such aspects. Studies by Petre and Green (Petre & Green, 1992) of engineers using CAD systems for designing computer circuits, demonstrated that the most significant difference between novices and experts is in the use of layout to capture domain information. In such circuit diagrams, the layout of components is not specified as being

semantically significant. Nevertheless, experienced designers exploit layout to carry important information, e.g. by grouping together components that are functionally related. By contrast, certain diagrams produced by novices were considered poor because they either failed to use layout or, in particularly ‘awful’ examples, were especially confusing through the misuse of common layout conventions informally adopted by experienced engineers (Gurr, 1999).

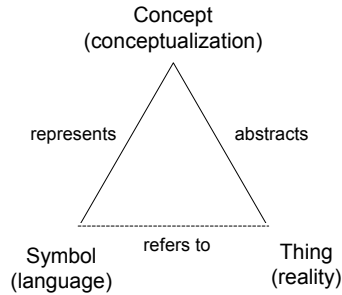
2.1.4 Real-World Semantics

One of the main success factors behind the use of a modeling language is its ability to provide to its target users a set of modeling primitives that can directly express relevant domain abstractions. Domain abstractions are constructed in terms of concepts, i.e., abstract representations of certain aspects of entities that exist in a given domain that we name here a *domain conceptualization*. An abstraction of a certain *state of affairs* expressed in terms of a set of domain concepts, i.e., according to a certain conceptualization, is termed a *model* in this work (Ferreira Pires, 1994; van Sinderen, 1995). Take as an example the domain of genealogical relations in reality. A certain conceptualization of this domain can be constructed by considering concepts such as *Person*, *Man*, *Woman*, *Father*, *Mother*, *Offspring*, *being the father of*, *being the mother of*, among others. By using these concepts, we can articulate a conceptual model of certain facts in reality such as, for instance, that a man named John is the father of another man named Paul.

Conceptualizations and Models are abstract entities that only exist in the mind of the user or a community of users of a language. In order to be documented, communicated and analyzed they must be captured, i.e. represented in terms of some concrete artifact. This implies that a language is necessary for representing them in a concise, complete and unambiguous way.

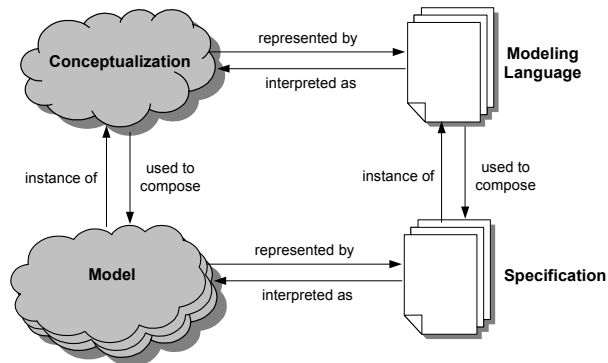
Figure 2.5 represents the relation between a language, a conceptualization and the portion of reality that this conceptualization abstracts. This picture depicts the well-known *Ullmann’s triangle* (Ullmann, 1972). This triangle derives from that of Ogden and Richards (Ogden & Richards, 1923) and from Ferdinand de Saussure (de Saussure, 1986), on whose theories practically the whole modern science of language is based.

Figure 2-5 Ullmann's Triangle: the relations between a thing in reality, its conceptualization and a symbolic representation of this conceptualization



The *represents* relation concerns the definition of language $\mathcal{L}$ real-world semantics. The dotted line between language and reality in this figure highlights the fact that the relation between language and reality is always intermediated by a certain conceptualization (Baldinger, 1980). This relation is elaborated in Figure 2.6 that depicts the distinction between a model and its representation, and their relationship with the conceptualization and representation language. In the scope of this work the representation of a model in terms of a representation language $\mathcal{L}$ is called a *model specification* (simply *specification*, or *representation*) and the language $\mathcal{L}$ used for its creation is called a *modeling* (or *specification*) *language*.

Figure 2-6 Relations between conceptualization, Model, Modeling Language and Specification



As previously mentioned, we defend the precedence of real-world concepts over formal concepts and implementational issues in the design of conceptual modeling languages. In particular, this thesis is not concerned with specific aspects of language technology such as formal syntax and semantics but with the so-called *domain appropriateness* and *comprehensibility appropriateness* of a given language (Krogstie, 2000; Halpin, 1998). In order for a specification S to faithfully represent a model $\mathcal{M}$, the modelling primitives of the language $\mathcal{L}$ used to produce S should faithfully represent the domain conceptualization C used to articulate the represented model

$\mathcal{M}$. The *domain Appropriateness* of language is a measure of the suitability of a language to model phenomena in a given domain, or in other words, of its truthfulness of a language to a given domain reality. On a different aspect, different languages and specifications have different measures of pragmatic adequacy. *Comprehensibility appropriateness* refers to how easy is for a user a given language to recognize what that language's constructs mean in terms of domain concepts and, how easy is to understand, communicate and reason with the specifications produced in that language.

The measures of these two quality criteria for a given language and domain are aspects of the *represents* relation depicted in figure 2.6. In the following section we elaborate on these aspects and present a framework to systematically evaluate the domain and comprehensibility appropriateness in accordance with a domain conceptualization. The *abstract* relation in figure 2.5 is discussed in chapter 3.

2.2 A Framework for Language Evaluation

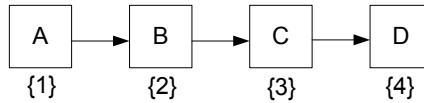
The purpose of the current chapter is to discuss the design and evaluation of artificial modeling languages for capturing phenomena in a given material domain according to a conceptualization of this domain. Before we aim at this target at a language level, i.e., at a level of a system of representations, we start discussing the simpler relation between a particular specification and a particular abstraction of a portion of reality, i.e., a particular model.

In (Gurr, 1998, 1999), the author presents a framework to formally evaluate the relation between the properties of a representation system and the properties of the domain entities they represent. According to him, representations are more or less effective depending on the level of homomorphism between the algebras used to represent what he terms the *representing* and the *represented* world, which correspond to the *specification* and *model* in our vocabulary, respectively.

Isomorphism is defined in mathematics as follows. Let A and B be two algebras, we say that a mapping between A and B is homomorphic iff the structure of relations over the elements of A is preserved by the relations which hold over the corresponding elements in B . However, if a homomorphism exists from A to B it does not mean that they are identical. It may be, for instance, that there are elements in B which are not mapped onto any element in A . Alternatively there can be individual elements in B which are mapped by to more than one element of A . If there is a homomorphic mapping between A and B and every element of B is mapped to by a unique element of A then the two algebras are identical and are said to be isomorphic (Gurr, 1999).

For example, consider the diagram depicted in figure 2.7 as a representation of four integers (e.g., from 1 to 4) and their ordering, where integers are represented by squares labeled with capital letters and the less-than relation is represented by the transitive closure of *is-arrow-connected*. In this case we may use the algebras $(\{1, 2, 3, 4\}, \{<\})$ and $(\{A, B, C, D\}, \{\text{transitive closure of } is\text{-arrow-connected}\})$ to describe the model and specification (diagram), respectively. One possible representational mapping between these entities maps 1-4 to A-D, respectively, and $<$ to the transitive closure of *is-arrow-connected*. The individuals in the model which are represented by each of the symbols A-D are shown between brackets in the picture 2.7. This kind of semantic correspondence (that also appears in figure 2.9, 2.11, 2.13, 2.14 and 2.16) is not part of the language itself and is shown in the pictures with the only purpose of enhancing clarity of our explanations.

Figure 2-7 A representation for the numbers domain



Gurr argues at length that the stronger the match between a model and its representing diagram, the easier is to reason with the latter. The easiest case is when these matches are isomorphisms. The implication of this for the human agent who interprets the diagram is that his interpretation correlates precisely and uniquely with an abstraction being represented. By contrast, where the correlation is not an isomorphism then there may potentially be a number of different models which would match the interpretation.

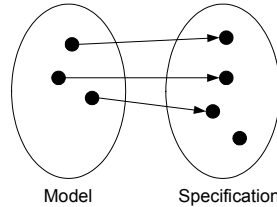
The evaluation framework proposed by Gurr focuses on evaluating the match between individual diagrams and the state of affairs they represent. In (Wand & Weber, 1989, 1990; Weber, 1997), another framework is defined for evaluating *expressiveness* and *clarity* of modeling grammars, i.e., with the focus on the system of representations as a whole. In our work, these two proposals are merged in one single evaluation framework. We focus our evaluation on the level of the system of representations. Nevertheless, as it will be shown in the following subsections, by considering desirable properties of the mapping of individual diagrams onto what they represent, we are able to account for desirable properties of the modeling languages used to produce these diagrams, extending in this way Wand & Weber's original proposal.

In (Gurr, 1999), four properties are defined, which are required to hold for a homomorphic correlation to be an isomorphism: *lucidity*, *soundness*, *laconicity* and *completeness*. These properties are discussed as follows.

2.2.1 Lucidity and Construct Overload

A specification $\mathcal{S}$ is called **lucid** with respect to (w.r.t.) a model $\mathcal{M}$ if a (representation) mapping from $\mathcal{M}$ to $\mathcal{S}$ is *injective*. A mapping between $\mathcal{M}$ and $\mathcal{S}$ is injective iff every entity in the specification $\mathcal{S}$ represents **at most** one (although perhaps none) entity of the model $\mathcal{M}$. An example of an injective mapping is depicted in figure 2.8.

Figure 2-8 Example of a Lucid representation mapping from Model to Specification



The notion of lucidity at the level of individual diagrams is strongly related to the notion of *ontological clarity* at the language level as discussed in (Weber, 1997; Wand & Wang, 1996; Wand & Weber, 1989). In (Weber, 1997), the author states that the ontological clarity of a modeling grammar is undermined by what they call **construct overload**: “*construct overload occurs when a single grammatical construct can stand for two or more ontological constructs, The grammatical construct is overloaded because it is being used to do more than one job.*”

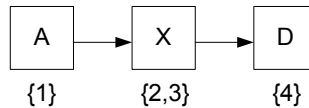
The notions of *lucidity* and *ontological clarity* albeit related are not identical. A construct can be overloaded in the language level, i.e. it can be used to represent different concepts, but every manifestation of this construct in individual specifications is used to represent only one of the possible concepts. An example of construct overload w.r.t. the conceptualization proposed in chapters 4 to 7 occurs in the *relation* construct in the *relational data model* (Codd, 1970), which is used both to represent *object types* (e.g. *Person*, *Car*, *Student*) and types whose instances are *mutual properties* shared by individual objects (e.g., *enrolment*, *marriage*). However, it is not the case that in a particular model the relation construct is used to represent a type whose instances are both objects and mutual properties. Figure 2.9 exemplifies a non-lucid representation. In this case, the construct **X** is used to represent two entities of the model, namely the numbers 2 and 3. In this case, although the representation system does not have a case of construct overload (since labeled boxes only represent numbers and arcs only represent the *less-than* relation between numbers) the resulting specification is non-lucid. In summary, the absence of construct overload in a language does not directly prevent the construction

of non-lucid representations in this language. Additionally, construct overload does not entail non-lucidity.

Nevertheless, non-lucidity can also be manifested at a language level. We say that a language (system of representation) is non-lucid according to a conceptualization if there is a construct of the language which is non-lucid, i.e., a construct that when used in a specification of a model (instantiation of this conceptualization) stands for more than one entity of the represented model. As demonstrated in chapter 6, the UML construct of an association class can be seen as a non-lucid construct (according to the conceptualization presented in that chapter) since it represents simultaneously a *mutual property* shared by a multitude of entities (e.g., *marriage*) and a *relation*⁴ (e.g., *being-the-husband-of*, *being-the-wife-of*, *being-married-to*) induced by this property. Non-lucidity at the language level can be considered as a special case of construct overload that does entail non-lucidity at the level of individual specifications.

Construct overload is considered an undesirable property of a modeling language since it causes ambiguity and, hence, undermines clarity. When a construct overload exists, users have to bring additional knowledge not contained in the specification to understand the phenomena which are being represented. For instance, in the relational data model example given above, users have to examine the relations to determine whether an object type or a mutual property type is being represented. A non-lucid representation language entails non-lucid representations which clearly violate the Gricean conversational maxim that requires contributions to be neither ambiguous nor obscure. In summary, a modeling language should not contain construct overload and every instance of a modeling construct of this language should represent only one individual of the represented domain abstraction.

Figure 2-9 Example of a Non-Lucid diagram



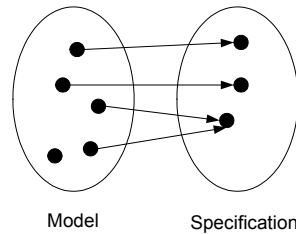
2.2.2 Soundness and Construct Excess

A specification $\mathcal{S}$ is called **sound** w.r.t. a model $\mathcal{M}$ if a (representation) mapping from $\mathcal{M}$ to $\mathcal{S}$ is *surjective*. A representation mapping from $\mathcal{M}$ to $\mathcal{S}$ is surjective iff the corresponding interpretation mapping from $\mathcal{S}$ and $\mathcal{M}$ is total, i.e. iff every entity in the specification $\mathcal{S}$ represents **at least** one entity

⁴ The notions of object type, mutual property (relator type) and relation among others are defined and discussed in depth in chapter 6 of this thesis.

of the model $\mathcal{M}$ (although perhaps several). An example of a surjective representation mapping is depicted in figure 2.10.

Figure 2-10 Example of a Sound representation mapping from Model to Specification



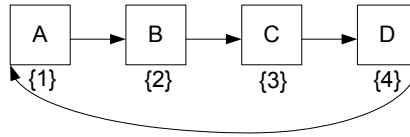
An example of an unsound diagram is illustrated in figure 2.11. The arc connecting the labeled boxes D and A does not stand for any relation in the represented world.

Unsoundness at the level of individual specifications is strongly related to unsoundness at language level, a property that is termed **construct excess** by Weber: “*construct excess occurs when a grammatical construct does not map onto an ontological construct*” (Weber, 1997).

In analyzing information system grammars in terms of an ontological theory, Weber advocates that the common notion of an *optional attribute* can be considered a case of construct excess. He claims that according to the theory used, there is no such a thing as an optional property. For instance, it is not the case that a person has the “*optional property*” of having children but, instead that there is a subtype of Person, say Parent, whose instances all have this property. Although construct excess can result in the creation of unsound specifications, soundness at the language level does not prohibit the creation of unsound specifications. For example, there is no construct excess in the language used to produce the specification of figure 2.11.

An unsound diagram violates the Gricean cooperative principle since any represented construct will be assumed to be meaningful by users of the language. Since no mapping is defined for the exceeding construct, its meaning becomes uncertain, hence, undermining the clarity of the specification. According to (Weber, 1997), users of a modeling language must be able to make a clear link between a modeling construct and its interpretation in terms of domain concepts. Otherwise, they will be unable to articulate precisely the meaning of the specifications they generate using the language. Therefore, a modeling language should not contain construct excess and every instance of its modeling constructs must represent an individual in the domain.

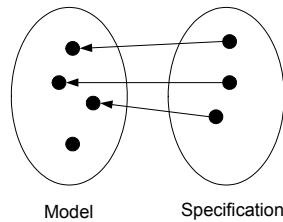
Figure 2-11 Example of an Unsound diagram



2.2.3 Laconicity and Construct Redundancy

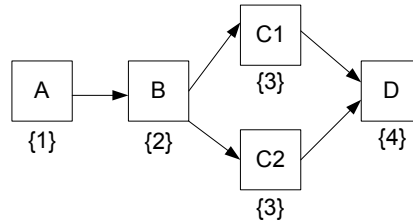
A specification S is called **laconic** w.r.t. a model $\mathcal{M}$ if the interpretation mapping from S to $\mathcal{M}$ is *injective*, i.e. iff every entity in the model $\mathcal{M}$ is represented by **at most** one (although perhaps none) entity in the representation S . An example of an injective interpretation mapping is depicted in figure 2.12. The notion of *laconicity* in the level of individual specifications is related to the notion of **construct redundancy** in the language level in (Weber, 1997): “*construct redundancy occurs when more than one grammatical construct can be used to represent the same ontological construct.*”

Figure 2-12 Example of a Laconic Interpretation mapping from Specification to Model



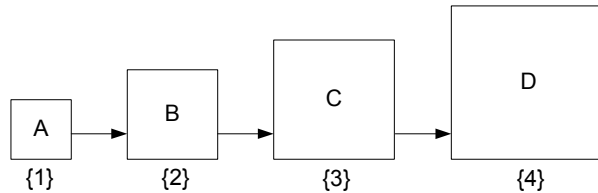
Once again, despite of being related, laconicity and construct excess are two different (even opposite) notions. On one hand, construct redundancy does not entail non-laconicity. For example, a language can have two different constructs to represent the same concept. However, in every situation the construct is used in particular specifications it only represents a single domain element. On the other hand, the lack of construct redundancy in a language does not prevent the creation of non-laconic specifications in that language. An example of a non-laconic diagram is illustrated in figure 2.13. In this picture, the same domain entity (the number 3) is represented by two different elements (C_1 and C_2) although the representation language used does not contain construct redundancy.

Figure 2-13 Example of a Non-Laonic diagram



Non-laonicity can also be manifested at the language level. We say that a language is non-laonic if it has a non-laonic modeling construct, i.e. a construct that when used in a specification of a model causes an entity of this model to be represented more than once. For instance, take a version of the labeled boxes language used so far and let the less-than relation between numbers be represented both as the *transitive closure of the is-arrow-connected* and by the *is-smaller-than* relation between labeled boxes. All specifications using this representation (e.g. figure 2.14) are deemed non-laonic. Non-laonicity at the language level can be considered as a special case of construct redundancy that does entail non-laonicity at the level of individual diagrams.

Figure 2-14 Example of a Non-Laonic diagram generated by a Non-Laonic language



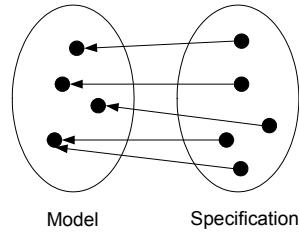
In (Weber, 1997), the authors claim that construct redundancy “adds unnecessarily to the complexity of the modeling language” and that “unless users have in-depth knowledge of the grammar, they may be confused by the redundant construct. They might assume for example that the construct somehow stands for some other type of phenomenon.” Therefore, construct redundancy can also be considered to undermine representation clarity. Non-laonicity also violates the Gricean principle, since redundant representations can be interpreted as standing for a different domain element. In summary, a modeling language should not contain construct redundancy, and elements in the represented domain should be represented by at most one instance of the language modeling constructs.

2.2.4 Completeness

A specification S is called *complete* w.r.t. a model $\mathcal{M}$ if an interpretation mapping from S to $\mathcal{M}$ is *surjective*. An interpretation mapping from S to $\mathcal{M}$ is surjective iff the corresponding representation mapping from $\mathcal{M}$ to S is

total, i.e., iff every entity in a model (instance of the domain conceptualization) is represented by *at least* one (although perhaps many) entity in the representation $\mathcal{S}$. An example of a *surjective* interpretation mapping is depicted in figure 2.15.

Figure 2-15 Example of a Complete Interpretation mapping from Specification to Model



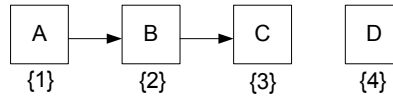
The notion of completeness at the level of individual specifications is related to the notion of *ontological expressiveness* and, more specifically, *completeness* at the language level, which is perhaps the most important property that should hold for a representation system. A modeling language is said to be *complete* if every concept in a domain conceptualization is covered by at least one modeling construct of the language. Language incompleteness entails lack of expressivity, i.e., that there are phenomena in the considered domain (according to a domain conceptualization) that cannot be represented by the language. Alternatively, users of the language can choose to overload an existing construct, thus, undermining clarity.

In chapter 8 of this thesis, when evaluating the Unified Modeling Language (UML) in terms of the conceptualization proposed in chapters 4 to 7, examples of incompleteness abound. To mention two cases, there are several different sorts of *object types* and *part-whole relations* in the conceptualizations proposed in chapter 4 and 5, respectively, which are not directly represented by any construct of the language. In both cases, the distinct concepts present in the conceptualization are overloaded by the language constructs of *class* and *aggregation/composition*, respectively.

An incomplete modeling language is bound to produce incomplete specifications unless some existing construct is overloaded. However, the converse is not true, i.e. a complete modeling language can still be used to produce incomplete specifications. An example of the latter is shown in figure 2.16. In this picture, a domain element (the $3 < 4$ relation) is not present in the representation. In accordance with the detailed account of Grice's cooperative principle (specifically, that all necessary information is included), specification and language designers should attempt to ensure completeness as most readers assume this to be true. In summary, a modeling language should be complete w.r.t. a domain conceptualization and every element in a domain abstraction (instance of this domain

conceptualization) must be represented by an element of a specification built using this language.

Figure 2-16 Example of an Incomplete diagram



2.3 Conceptualization and Ontology

In section 2.2, we advocate that the suitability of a language to create specifications in a given domain depends on how “close” the structure of the specifications constructed using that language resemble the structure of the models (domain abstractions) they are supposed to represent. To put it more technically, a specification S produced in a language $\mathcal{L}$ should be, at least, a homomorphism of the model $\mathcal{M}$ that S represents.

The framework presented in section 2.2 is ultimately based on the analysis of the relation between the structure of a modeling language and the structure of a domain conceptualization.

What is referred by *structure of a language* can be accessed via the description of the specification of *conceptual model underlying the language*, i.e., a description of worldview embedded in the language’s modeling primitives. In (Milton & Kamierczak, 2004), this is called the *ontological metamodel of the language*, or simply, the *ontology of the language*. From a philosophical standpoint, this view is strongly associated with Quine (Quine, 1969), who proposes that an ontology can be found in the *ontological commitments* of a given language, that is, the entities the primitives of a language commit to the existence of. For example, Peter Chen’s Entity Relationship model (Chen, 1976) commits to a worldview that accounts for the existence of three types of things: *entity*, *relationship* and *attribute*.

This idea can be understood in analogy to the distinction between a conceptual model and design model in information systems and software engineering. Whilst the former is only concerned with modeling a view of the domain according to a given application, the latter is committed to translating the model of this view on the most suitable implementation according to the underlying implementational environment and also considering a number of non-functional requirements (e.g., security, fault-tolerance, adaptability, reusability, etc.). Likewise, the specification of the conceptual model underlying a language is the description of what the *primitives of a language* are able to represent in terms of real-world phenomena. In some sense (formally characterized in chapter 3), it is the representation of a conceptualization of the domain in terms of the

language's vocabulary. In the *design* of a language, these conceptual primitives can be translated into a different set of primitives. For example, it can be the case that a conceptual primitive is not directly represented in the actual abstract syntax of a language, but its modeling capabilities (the real world concept underlying it) can be translated to several different elements in the language's abstract syntax due to non-functional requirements (e.g., pragmatics, efficiency). Nonetheless, the design of a language is responsible for guaranteeing that the language's syntax, formal semantics and pragmatics are conformant with this conceptual model. From now on, the *Modeling Language* icon depicted in figure 2.6 represents the specification of the conceptual model underlying the language, or what we shall name the **language metamodel specification**.

The *structure of domain conceptualization* must also be made accessible through an explicit and formal description of the corresponding portion of reality in terms of a concrete artifact, which is termed here a *domain reference ontology*, or simply, a **domain ontology**. The idea is that a reference ontology should be constructed with the sole objective of making the best possible description of the domain in reality w.r.t. to a certain level of granularity and viewpoint. Additionally, it should be constructed using formal and methodological tools which have been developed by the area of *formal ontology* (Husserl, 1970) in philosophy. Finally, it should be application independent and not be biased towards a specific mathematical model or formal theory.

The notion of ontology as well as its role in the explicit representation of conceptualizations is discussed in depth and given a formal characterization in chapter 3.

2.3.1 Designing a Visual Modeling Language for the Domain of Genealogy: An Example

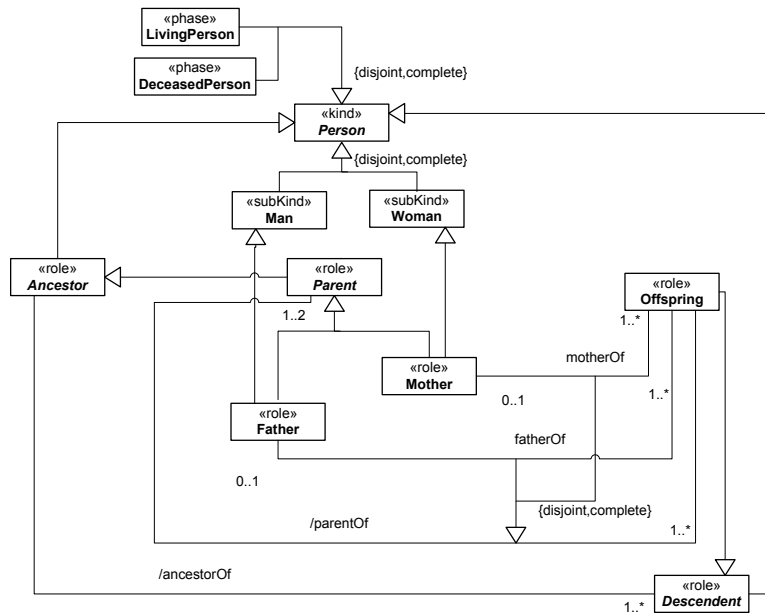
In the sequel we illustrate these rather abstract notions discussed so far with an example in the domain of genealogical relations. This domain is simple and familiar and, hence, can be useful for illustration purposes. Nonetheless, it is still not a completely artificial example since there are, in practice, standard modeling languages to represent genealogical relations and a number of applications that make use of these standards (Howells, 1998). A certain conceptualization of this domain can be articulated by considering concepts such as *person*, *living person*, *deceased person*, *father*, *mother*, *offspring*, *fatherOf* and *motherOf*. These concepts are related to each other and have their interpretation constrained by axioms imposed on their definitions. Figure 2.17 depicts a domain ontology representing a possible conceptualization in this domain. The modeling primitives of the *UML profile* (OMG, 2003b) used to represent

this ontology are part of the ontology representation language proposed throughout this work. The class stereotypes present in this specification (*kind*, *subkind*, *role* and *phase*), in particular, are discussed in depth in chapter 4.

The diagram in this picture is complemented by the following axioms:

- A person x is a *parentOf* person y iff x is *fatherOf* y or x is *motherOf* y ;
- A person x is a *ancestorOf* person y iff x is *parentOf* y or there is a person z such that z is an *parentOf* y and x is *ancestorOf* z ;
- A person cannot be its own ancestor (i.e., the *ancestorOf* relation is irreflexive);
- If a person x is an *ancestorOf* person y then y cannot be an *ancestorOf* x (i.e., the *ancestorOf* relation is asymmetric);
- If a person x is an *ancestorOf* person y and y is an *ancestorOf* person z then x is an *ancestorOf* z (i.e., the *ancestorOf* relation is transitive).

Figure 2-17 An Ontology for the Genealogy Domain



By representing a conceptualization of this domain in terms of this concrete artifact we can design a language to express phenomena in this domain capturing characteristics that this conceptualization deems relevant. For instance, according to this domain ontology, *Person* is an abstract type, i.e., one that cannot have direct instances. This is represented in this notation

by depicting the name of type in italics. The abstract type *Person* is partitioned in two independent suptyping structures:

1. **Man, Woman:** this partition represents that every individual person (instance of type *Person*) in the universe of discourse is either a man (an instance of *Man*) or woman (instance of *Woman*). Moreover, due to the «subKind» stereotype, it states that every man is *necessarily* a man (in the modal logics sense), i.e., every instance of the type man is a man in every possible situation considered by the model. We say in this case that *Man* is a *rigid* classifier (see chapter 4). *Mutatis Mutandis* the same applies to instances of the type *Woman*. Finally, it states that both individual man and woman obey a *principle of identity* supplied by the type *Person* (due to the presence of the «kind» stereotype). The notion of a principle of identity is discussed in depth in chapter 4. For now, we can say that every instance of *Person* maintains its identity (i.e., it is the same *Person*) in every circumstance considered by the model;
2. **LivingPerson, DeceasedPerson:** this partition represents that every individual person in the universe of discourse is either a living person or a deceased one. However, in contrast to the **⟨Man, Woman⟩** partition, an instance of *LivingPerson* is not necessarily so (in the modal sense), i.e., *LivingPerson* is a *anti-rigid* classifier (see chapter 4). Every instance of *LivingPerson* is only *contingently* an instance of *LivingPerson* (again in the modal sense). That is to say that for every *x* such that *x* is *LivingPerson* there is a counterfactual situation in which *x* is not a *LivingPerson*, which in this case, implies that *x* is a *DeceasedPerson* in this counterfactual situation. One more, *Mutatis Mutandis*, the same applies to instances of *DeceasedPerson*. These facts are implied by the presence of «phase» stereotyped classifiers and the associated constraint that they must be defined in a partition.

A cross-relation of these two partitions give us four concrete classifiers, i.e., classifiers that can have direct instances, let us name them *LivingMan*, *DeceasedMan*, *LivingWoman* and *DeceasedWoman*. Every instance of person in a given situation is necessarily an instance of one of these classifiers. A suitable modeling language must have modeling primitives that conform to these constraints. Other constraints of the possible models according to this ontology include: that every *offspring* can have at maximum one *father* and one *mother*; the *ancestorOf* relation (defined to hold between instances of *Person*) is irreflexive, antisymmetric and antitransitive. The set of constraints captured in this ontology represents part of the conceptualized *structure of*

the domain. This structure must be taken into account in the design (an evaluation) of a language to model genealogical relations.

By having a conceptualization (abstract entity) represented in terms of a domain ontology, and by applying the framework discussed in section 2.2, one can, in a precise manner, design a suitable modeling language for that given domain. In this particular case, we are able to design a language $\mathcal{L}_1$ whose metamodel specification $S_{\mathcal{L}_1}$ is isomorphic to the ontology of figure 2.17. The primitives of this language are presented in figure 2.18 below.

Figure 2-18 Domain Concepts and their representing modelling primitives in $\mathcal{L}_1$

Language $\mathcal{L}_1$									
Ontology	Living Man	Deceased Man	Living Woman	Deceased Woman	ParentOf	Father	Mother	Offspring	AncestorOf

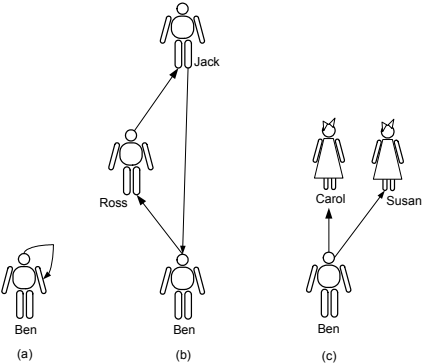
A number of characteristics should be observed about language $\mathcal{L}_1$. To start with, the language contains modeling primitives that represent all concrete classifiers (*LivingMan*, *DeceasedMan*, *LivingWoman*, *DeceasedWoman*) and non-derived relations present in the ontology (*fatherOf*, *motherOf*). Consequently, we can say that the language is expressive enough to represent all characteristics of the domain that are considered relevant by the underlying ontology. Moreover, in the mapping from ontology O of figure 2.17 to the metamodel specification $S_{\mathcal{L}_1}$ (of language $\mathcal{L}_1$) there is no case of construct redundancy, construct overload or construct excess.

In regards to the property of completeness, when mapping the elements of a domain ontology to a language metamodel specification we must guarantee that these elements are represented in their full formal descriptions. In other words, the language metamodel specification $S_{\mathcal{L}_1}$ representing the domain ontology O of figure 2.17 must also represent in its well-formedness rules this ontology's full axiomatization. In formal, model-theoretic terms, this means that these entities should have the same set of logical models. In chapter 3, we discuss this topic in depth and present a formal treatment of this idea. The set of logical models of O represent the state of affairs in reality deemed possible by a given domain conceptualization. In contrast, the set of logical models of S stand for the

world structures which can be represented by the grammatically correct specifications of language $\mathcal{L}_1$. In summary, we can state that if a domain ontology O is fully represented in a language metamodel specification $\mathcal{S}_{\mathcal{L}_1}$ of $\mathcal{L}_1$, then the only grammatically correct models of $\mathcal{L}_1$ will be those which represent state of affairs in reality deemed possible by the domain conceptualization represented by O .

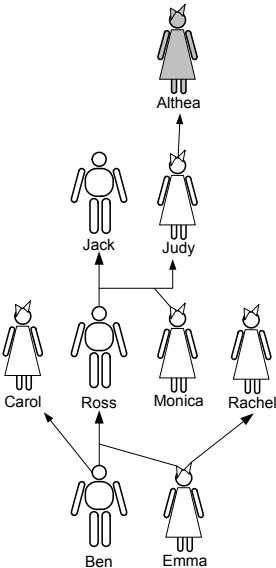
In this example, since we assume that O is fully represented in $\mathcal{S}_{\mathcal{L}_1}$, as a consequence, the specifications depicted in figures 2.19.a, 2.19.b and 2.19.c cannot be considered syntactically valid specification of language $\mathcal{L}_1$.

Figure 2-19 Examples of grammatically invalid models in language $\mathcal{L}_1$



In contrast, a valid specification in language $\mathcal{L}_1$ is depicted in 2.20 below.

Figure 2-20 Example of a valid model in language $\mathcal{L}_1$



Another aspect that should be noticed is how the ontology of figure 2.17 contributes for improving pragmatic efficiency in the concrete syntax of $\mathcal{L}_1$. In (Gurr, 1999), when introducing the framework that has been adapted in section 2.2, the author uses regular algebraic structures to model a domain conceptualization. We claim that many additional benefits arise from a more complete representation of the domain conceptualization than the algebra used in his work. We defend the idea that the more we know about a domain the better we can design pragmatically effective languages. In particular, there are important meta-properties of domain entities (e.g., *rigidity*, *relational dependency*) that are not captured by ontologically-neutral mathematical languages (see chapter 3, section 3.4.2). The failure to consider these meta-properties hinders the possibility of accounting for other direct aspects of visual syntaxes. In the case of language $\mathcal{L}_1$, one can observe that:

1. The types *Man* and *Woman* are kinds and, thus, *rigid types*, which means that instances of these types will continue to be so as long as they exist in the model. In contrast, an individual man (or woman) can have the (intrinsic) properties of *begin alive* or *being dead* in different situations. In any case, the man which is alive in one circumstance and dead in another is the same man, i.e., he maintains his identity across situations. In language $\mathcal{L}_1$, the icons⁵ used to represent instances of Person maintain the stable visual percept, which represents the dichotomy of the rigid types *Man* and *Woman*. The phases living and deceased are represented as variations of this visual percept, that is, the *same* visual percept can appear in different situations as one of the two variations;
2. The types modeled as concrete *Roles* in the ontology are *Father*, *Mother* and *Offspring*. These types are not only *anti-rigid* but also *relationally dependent*, i.e., the *same* instance x of *Father* can exist in another situation in the model without being a *Father*. Moreover, to be a *Father* is to be a *Man* who has (at least) one *Offspring*, i.e., for x to be a *Father* he must share a relational property with another individual who is an instance of *Offspring*. In $\mathcal{L}_1$, the *Parent* role is represented by the adjacency *relation* between the icon representing a Person and the arrow-head of the symbol representing the *parentOf* relation. Additionally, the *Offspring* role is represented by the adjacency relation between the icon representing a Person and the arrow-tail of the symbol representing the *parentOf* relation. This representation choice highlights the dependency of these roles on relations, which in turn, stand for mutual properties

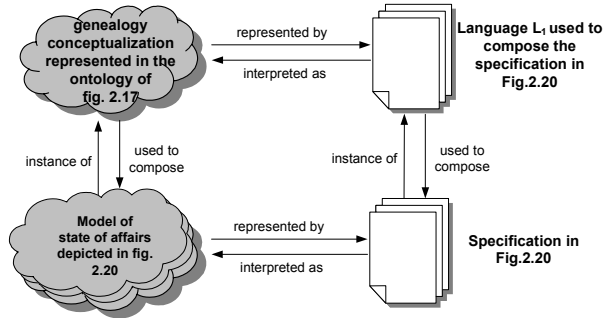
⁵ Icon is used here in the Peircean sense, i.e. as a sign that physically resembles what it ‘stands for’ (Peirce, 1960).

shared by their relata. Moreover, it allows for the representation that *x qua Man* maintains its identity in the scope of different relations and across different situations.

3. The *ancestorOf* relation is represented by the above relation in the plane associated with the *arrow-path-connectedness*, i.e., if *x* and *y* are two persons that are path-connected and *x* is above *y* in the plane then *x* is an *ancestorOf* *y*. The composed relation *above-path-connect* is also irreflexive, asymmetric and transitive, i.e., a strict partial order relation. These are exactly the same meta-properties as enjoyed by the *ancestorOf* relation. For this reason, the conclusion that (*x ancestorOf* *z*) if (*x ancestorOf* *y*) and (*y ancestorOf* *z*) is directly inferred from (*x* is *above-path-connected-to* *z*) if (*x* *above-path-connected-to* *y*) and (*y* *above-path-connected-to* *z*).

By instantiating the pattern of figure 2.6 to this domain we obtain the correspondence depicted in figure 2.21. The ontology *O* of figure 2.17 is a concrete representation of a given conceptualization of the genealogy domain. In this case, we have the ideal situation that the metamodel of language $\mathcal{L}_1$ is identical to this ontology. The genealogy concepts represented in *O* are used to articulate models of individual state of affairs in reality. A specification in language $\mathcal{L}_1$ (such as the one of figure 2.20) is a concrete artifact representing one of these models.

Figure 2-21 Instantiating the pattern of figure 2.6 for the domain of genealogy



In this example, the suitability of language $\mathcal{L}_1$ is evaluated w.r.t. a specific conceptualization and, more precisely, w.r.t. a specification of this conceptualization captured in the domain ontology of figure 2.17. A question that one may ask is how to evaluate the adequacy of a conceptualization and the corresponding ontology? In other words, how good is a conceptualization as an abstraction of a given universe of discourse or portion of reality (see *abstracts* relation in figure 2.5) and how good is an ontology as a representation of this conceptualization. The construction of a

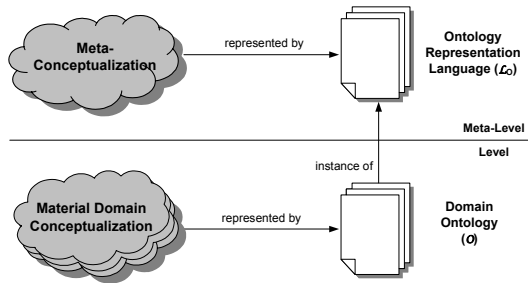
particular conceptualization, i.e. a particular shared abstraction of reality, should be based on the consensus of a community of domain experts, and could be supported by both theoretical and empirical evidence. An ontology, conversely, is a concrete artifact which is developed by an engineering process (Gómez-Pérez, Fernández-López, & Corcho, 2004), which includes the activities of requirements specification and evaluation (among others). We shall resume the discussion about these issues in chapter 3.

2.3.2 Meta-Conceptualization and General Conceptual Modeling Languages

A domain such as the genealogy discussed in previous section is what we have previously named a *material domain* (Smith, 1989; Little, 2003) and a language, such as $\mathcal{L}_1$, designed to model phenomena in this domain is called a *domain-specific language*. The design of domain-specific languages is a current and important research topic (Kelly & Tolvanen, 2000; Tolvanen, Gray & Rossi, 2004; Bottoni & Minas, 2003) whose main benefits can be summarized by the following quote from Tolvanen, Gray and Rossi (ibid.): “A [domain-specific language] raises the level of abstraction, while at the same time narrowing down the design space. The language follows the domain abstractions and semantics, allowing developers to perceive themselves as working directly with domain concepts. Industrial experiences of this approach show major improvements in productivity, time-to-market responsiveness and training time.”

In this work, however, the focus is not on designing or evaluating domain-specific languages. Contrariwise, one of the objectives is to propose a domain independent language, such as the UML profile used in figure 2.17, which can be used to represent domain ontologies in different material domains. Since a domain ontology is also a concrete artifact, it must be represented in some specification language $\mathcal{L}_0$. To be consistent with the position defended here, the language $\mathcal{L}_0$ used to represent individual domain ontologies should also be based on a domain conceptualization, in this case, a meta-conceptualization. This idea is depicted in figure 2.22.

Figure 2-22 Relations between a Material Domain Conceptualization, Domain Ontology, General Meta-Conceptualization and Ontology Representation Language



Since we consider here a domain ontology as a special type of conceptual model specification, we also consider an *ontology representation language* as a special type of *general conceptual modeling language*. From now on, we simply use the term *conceptual modeling languages* to refer to the class of domain-independent languages that are used to create conceptual specifications in material domains.

With regard to figure 2.22 some questions that come to the mind are: What kinds of entities compose a meta-conceptualization? How to construct a suitable meta-conceptualization that a general conceptual modeling language should commit to? As we discuss in chapter 3 of this thesis, in the specific case of existing conceptual modeling and ontology representation languages, their design has been strongly influenced by goals other than domain appropriateness. Some of these languages have been created in an ad hoc way, in a manner which is governed much more by intuition and generalization of specific practical cases than based on a deep analysis of the underlying reality. Others have been purposefully biased by design and implementation issues or in favor of mathematical models that simplify the definition of the formal semantics of the language. Examples include conceptual modeling languages that only account for concepts that can be represented in current database of programming technology or, like most knowledge representation languages, which only consider concepts that can be treated efficiently in automatic inference process.

The position strongly advocated here is that a suitable conceptual modeling language should commit to a general (i.e., domain independent) theory of real-world categories that account for the ontological distinctions underlying language and cognition. This is exactly the business of the branch of philosophy called Formal Ontology and, in particular, descriptive metaphysics. The term Formal Ontology has been coined by Edmund Husserl (Husserl, 1970) as an analogy to Formal Logic. Whilst Formal Logic deals with formal logical structures (e.g. truth, validity, consistency), Formal Ontology deals with formal ontological structures (e.g. theory of parts, theory of wholes, types and instantiation, identity, dependence,

unity) which apply to all domains. The term Formal here is, thus, less related to the sense of precise, mathematical and more related to its more ancient meaning, namely, of something that obtains in all material spheres of reality.

In summary, while domain conceptualizations and, consequently, *domain ontologies* are established by the consensus of a community of users w.r.t. a material domain, a conceptual modeling language that can be used to express these domain ontologies must be rooted in a domain independent philosophically and cognitively well-founded system of real-world categories, i.e. a *foundational (upper-level)* ontology (Schneider, 2003a; Masolo et al., 2003a; Heller & Herre, 2004).

These issues are discussed in detail in chapter 3.

2.4 Ontology-Based Semantics and Language Comparability

In the discussions carried out so far we have considered just one aspect of the relation between ontology and language. We have proposed that a *reference ontology* should be constructed as the best possible representation of a conceptualization, with the sole purpose of being a truthful representation of the domain in reality. Therefore, a modeling language in a domain can be designed (or evaluated) for domain appropriateness and comprehensibility appropriateness via the harmonization of the worldview underlying the language (i.e. the ontological metamodel of the language) with the one described by the domain ontology.

However, there is another scenario in which reference ontologies can play an important role, namely, when comparing models produced by different languages whose domain conceptualizations overlap. Suppose we have two languages $\mathcal{L}_1$ and $\mathcal{L}_2$ that commit to the conceptualizations C_1 and C_2 , respectively. Now supposed that C_1 and C_2 can be combined in a more general conceptualization C . Assume the two languages have been developed to be used in different problem-solving tasks and intend to represent different (aspects of) entities of the domain conceptualization C . As a consequence, the languages have different syntaxes and semantics. One question that arises is: how to relate (compare, translate) specifications written in $\mathcal{L}_1$ and $\mathcal{L}_2$?

Since they have different syntaxes and semantics, $\mathcal{L}_1$ and $\mathcal{L}_2$ should not be compared only in syntactical terms. Languages can have equal syntaxes with radically different semantics as much as they can have different syntaxes with equivalent mappings to a common semantic domain. For the

case of automatic translation, the formal semantics of $\mathcal{L}_1$ and $\mathcal{L}_2$ cannot guarantee consistency in the results either. For example, if C is a (alternative) conceptualization that refers to a domain of family relations, the expression (x related-to y) in $\mathcal{L}_1$ can be wrongly translated to an expression `brother(a,b)` in $\mathcal{L}_2$, since the *related-to* relation satisfies all the formal axioms of brotherhood (irreflexive, symmetric, transitive).

Semantic comparability is a challenge in enterprise engineering due to the lack of interoperability among different (yet coexistent) process models. Many manufacturing engineering and business software applications use process information, including manufacturing simulation, production scheduling, manufacturing process planning, workflow, business process reengineering, product realization process modeling, and project management. Since each of these applications utilizes process information in a different way, process information is also represented differently in each application. This problem is even more critical, since enterprise systems must manage the heterogeneity inherent to its various sub-domains, by integrating different models into coherent frameworks (e.g., enterprise models for processes, structure, goals, deontic assignments).

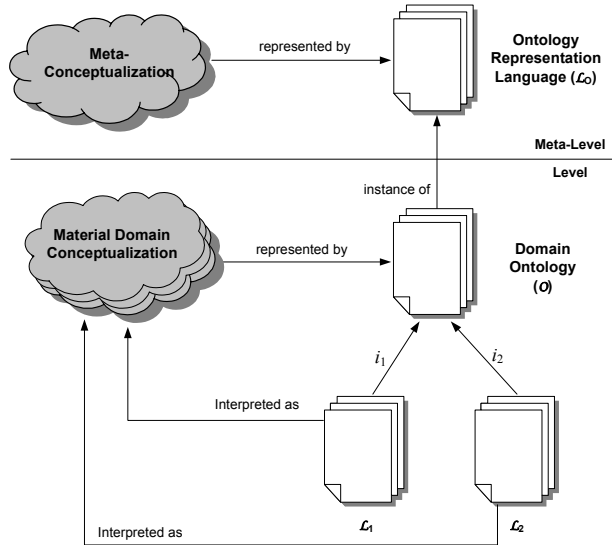
This problem has been currently addressed by the NIST PSL (Process Specification Language) Project⁶ and the community working on the Semantic Web (Euzenat, 2001). For instance, (Ciocoiu & Nau, 2000) provide a formal definition of what they term *ontology-based semantics* for enabling automated model translation between different first-order declarative languages. Their proposal is that, for a domain-specific language $\mathcal{L}$, a domain ontology O can serve as a specification of the implicit assumptions of the conceptualization underlying $\mathcal{L}$. If a mapping from the constructs of $\mathcal{L}$ to formulas in O can be defined, this mapping can, in turn, be used to derive the (ontology-based) semantics of $\mathcal{L}$. In simple terms, the ontology-based semantics of a specification S in language $\mathcal{L}$ is defined in terms of a set of logical models U of S such that U has the desired property of obeying the constraints imposed by ontology O . Finally, the notion of ontology-based models is used to define translations between a specification S_1 in language $\mathcal{L}_1$ and a specification S_2 in language $\mathcal{L}_2$, namely, S_2 is an *ontology-based partial translation* of S_1 iff every ontology-based model of S_1 is also an ontology-based model of S_2 . Then, a specification S_2 in language $\mathcal{L}_2$ is an *ontology-based translation* of specification S_1 in language $\mathcal{L}_1$ iff S_2 is an ontology-based partial translation of S_1 and S_1 is an ontology-based partial translation of S_2 .

⁶<http://ats.nist.gov/psl/>

In (Ciocoiu & Gruninger, 2000), this approach is exemplified via the translation between process models written in ILOG and IDEF3. This idea is illustrated in Figure 2.23. The solution creates logical interpretations i_1 and i_2 that relate the constructs of $\mathcal{L}_1$ and $\mathcal{L}_2$ to a common domain ontology O . The assumption here is that O is general enough so that its underlying conceptualization subsumes the parts of a conceptualization C targeted by $\mathcal{L}_1$ and $\mathcal{L}_2$. More than that, O is a formal logic theory through which the semantics of the conceptualization shared by $\mathcal{L}_1$ and $\mathcal{L}_2$ can be defined. Unlike the conceptual metamodels of $\mathcal{L}_1$ and $\mathcal{L}_2$, O is not biased towards a selection of concepts in C for the solution of specific tasks. It is a specification of domain-knowledge perceived as important by a community in order to address the needs of possibly several different applications.

We shall no longer discuss the topic of ontology-based translations in this thesis. The reader should refer to (Ciocoiu & Nau, 2000) for formal definitions of the approach just described, and to (Ciocoiu & Gruninger, 2000) for an interesting exemplification of these ideas.

Figure 2-23 Defining the Semantics of two Domain-Specific Languages via a shared Domain Ontology



2.5 Final Considerations

In this chapter we discuss the relation between a conceptual modeling language and the domain in reality that this language is supposed to

represent. We propose that the *domain appropriateness* and *comprehensibility appropriateness* of a modeling language can be systematically evaluated by comparing a concrete representation of the worldview underlying the language captured in the *language metamodel specification*, with an explicit and formal representation of a conceptualization, or a *reference ontology*. We advocate that in the best case, these two models are isomorphic and, hence, we propose a framework comprising of a number of properties (*lucidity*, *soundness*, *laconicity*, *completeness*) that must be reinforced for this isomorphism to take place. This framework combines the proposals of (Wand & Weber, 1989, 1990; Weber, 1997), which aim at the evaluation of representation systems, and (Gurr, 1998, 1999), which focus on the evaluation of individual representations.

If described formally, ontologies can also be beneficial as axiomatized theories through which the semantics of modeling languages can be defined for the purpose of semantic model interoperation (e.g., integration, comparison, translation). Additionally, if described in a language that commits to a suitable meta-conceptualization, ontologies can play an important role in informing properties that support the design of pragmatically efficient systems of visual concrete syntax.

The discussion carried out in this chapter encompasses both the level of material domains and corresponding domain-specific modeling languages, and the (meta) level of a domain-independent (meta) conceptualization that underpins a general conceptual (ontology) modeling language. The idea of maintaining a discussion in general terms was intended to show that the evaluation framework and the principles discussed here can be applied to the design of conceptual modeling languages irrespective of the generalization level to which they belong.

Nevertheless, due to the objectives and scope of this thesis, in the following chapters our discussion will focus on the level of meta-conceptualizations and of general conceptual modeling languages. Although the proposals made here contribute to the area of domain-specific languages design methodologies (as acknowledged, for instance, in Girardi & Serra, 2004), the focus of this thesis is certainly not on the design of this class of languages, neither with domain conceptualizations nor domain ontologies.